Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- **Veritabanı Tasarım Aşamaları**
  - Gereksinimlerin toplanması ve analiz edilmesi
    - Veri gereksinimleri: Ne gibi bilgiler saklanacak
    - İşlevsel gereksinimler: Ne gibi işlemler yapılacak (fiil noktası)
  - Kavramsal şema tasarımı: Varlık setleri, bağıntılar ve kısıtlamalar
  - Üst yüzeysel veri modelinden gerçekleştirme veri modeline dönüşüm
  - Mantıksal tasarım ve fiziksel tasarım ayrımı
    - Dersin konusu mantıksal tasarım
    - Fiziksel tasarım günümüzde çoğunlukla otomatik yapılmaktadır
- **Varlık Seti (Entity Set) Kavramı**
  - Aynı özelliğe sahip varlıklardan oluşan küme
  - Küçük dünyanın özneleri
  - Örnekler: Öğrenciler, işçiler, bölümler, dersler
  - Her özne varlık seti olmaz; varlıklar listeleniyor, sorgulanıyor ve takip ediliyorsa varlık seti oluşturulur
  - "Üniversite" tek bir üniversite veritabanında varlık seti olmaz, ancak Türkiye'deki tüm üniversiteler takip ediliyorsa varlık seti olur
- **Bağıntı (Relationship) Kavramı**
  - Varlık setlerinin birbirleriyle olan alakaları
  - Bağıntı kümesi: Birden çok ikili içeren küme
  - Bağıntı derecesi: İlişkili varlık sayısı (ikili = derecesi 2)
  - Bağıntı nitelikleri: Bağıntıya ait nitelikler (örn. task_assignment, start_date)
  - Bağıntı isimlendirmesi: Anlamlı isim seçimi önemli
- **Cardinality (Eleman Sayısı) Oranları**
  - Bire birlik (1:1)
  - Bire enlik (1:N)
  - Ene enlik (N:N)
  - Ene birlik (N:1)
  - Cardinality belirleme yöntemi: Diğer varlıklardan birer numune alıp cümle kurarak düşünmek
  - Ezbere değil, küçük dünyayı anlayarak belirlemek gerekir
- **Katılım (Participation) Kısıtlaması**
  - Partial (Kısmi): Varlık setindeki bazı varlıklar bağıntıda yer alır (tek çizgi)
  - Total (Tam): Varlık setindeki tüm varlıklar bağıntıda yer almak zorundadır (çift çizgi)
  - Örnek: Her departmanın mutlaka bir yöneticisi olmalı → total participation
- **Nitelik Türleri ve Notasyonları**
  - Anahtar (Key): Altı çizgili, unique/biricik nitelik
  - Çoklu değerli (Multi-valued): Çift çizgi, birden fazla değer alabilen nitelik
  - Kompozit (Composite): Birden çok alt niteliğe ayrılan nitelik
  - Derived (Türetilmiş): Fiziksel olarak saklanmayan, çıkarımla hesaplanan nitelik (kesik çizgi)
    - Örnek: Yaş bilgisi → doğum tarihinden hesaplanır
    - Örnek: Number_of_employees → SELECT COUNT ile hesaplanır
    - Saklanmasının yük getireceği durumlarda derived tercih edilir
- **Zayıf Varlık Seti (Weak Entity Set)**

  - Kendi başına anahtarı olmayan varlık seti
  - Güçlü bir varlık setiyle bire-bir veya en-bire var olma bağımlılığı kurmalı
  - Ayırt edici nitelik (discriminating attribute): Güçlü varlığın anahtarı altında zayıf varlıkları ayırt eden nitelik
  - Belirleyici bağıntı (identifying relationship): Çift çizgi ile gösterilir
  - Zayıf varlığın anahtarı = Güçlü varlığın birincil anahtarı + Ayırt edici nitelik
  - Örnekler:
    * Lise-öğrenci: Öğrenci no tek başına biricik değil, lise ismiyle birlikte biricik
    * Satış formu-satır: Satır no tek başına biricik değil, satış no ile birlikte biricik
    * Bina-oda: Oda no tek başına biricik değil, bina no ile birlikte biricik
    * Dependent (bağımlı aile üyeleri): İsim, bir işçinin çocukları arasında ayırt edici
- **Anahtar Kavramları**
  - Super Key: Tüm nitelikler bir arada varlıkları ayırt eder (fazla bile)
  - Minimal Super Key: Ayırt etme için gereken en az nitelik grubu
  - Aday Anahtar (Candidate Key): Minimal super key'lerden her biri
  - Birincil Anahtar (Primary Key): Aday anahtarlardan seçilen bir tanesi
  - Zayıf varlıkta anahtar bulunamazsa → zayıf varlık seti
- **Bağıntı Setlerinde Anahtar Belirleme**
  - Ene enlik bağıntıda: İki tarafın anahtarlarının hepsi birlikte anahtarı oluşturur
  - Bire enlik bağıntıda: En (N) tarafının anahtarı, bağıntı setinin anahtarıdır
  - Bire birlik bağıntıda: Herhangi bir tarafın anahtarı yeterli
  - Üçlü bağıntılarda: Cardinality'ye göre değişir
    * Her üç taraf çoklu ise → hepsi birlikte anahtar
    * Bir taraf bir ise → diğer ikisinin kombinasyonu anahtar olabilir
    * Tek bir nitelik tek başına yeterli olamaz
- **Çok Dereceli (Ternary) Bağıntılar**
  - Üç varlık seti arasında bağıntı
  - Örnek: İşçi-Proje-Kabiliyet (Skill): Bir işçi bir projede bir yeteneğini kullanıyor
  - Üçlü bağıntının ikili bağıntılara dönüştürülmesi:
    * Bazı durumlarda mümkün, bazı durumlarda değil
    * Zayıf varlık kullanarak üçlü bağıntı ikiliye dönüştürülebilir
    * Üç güçlü varlığın anahtarları birleşerek zayıf varlığın anahtarını oluşturur
- **Recursive (Özyinelemeli) Bağıntılar**
  - Aynı varlık seti üzerinde bağıntı
  - Örnek: Employee-Employee arasında yönetme (manage) bağıntısı
  - Rol tanımlama: Manager rolü, Subordinate rolü
  - Örnek: Bir işçinin bir yöneticisi var (1), bir yöneticinin en tane astı var (N)
- **Genelleştirme ve Özelleştirme (Generalization / Specialization)**
  - Super type (süper varlık) → Sub type (alt varlık) ilişkisi
  - Super type'ın tüm nitelikleri sub type'lar için geçerli
  - Sub type'ların kendine özgü nitelikleri olabilir
  - Kısıtlamalar:
    * Disjoint (Ayrık): Bir varlık sadece bir alt tipe ait olabilir (OR)
    * Overlap (Örtüşme): Bir varlık birden fazla alt tipe ait olabilir (AND)
    * Total participation: Tüm varlıklar mutlaka bir alt tipe ait olmalı (çift çizgi)
    * Partial participation: Bazı varlıklar alt tiplerden hiçbirine ait olmayabilir
  - Dört kombinasyon: Total-Disjoint, Total-Overlap, Partial-Disjoint, Partial-Overlap
  - Örnekler:
    * Gemi → Yolcu gemisi, Yük gemisi (total, disjoint)
    * Employee → Manager, Engineer, Technician, Secretary (partial, disjoint)
    * Employee → Employee, Customer (total, overlap)
- **Aggregation (Kümeleme)**
  - Üçlü bağıntıya alternatif olarak tanımlanan kavram
  - Bir bağıntı üzerine başka bir bağıntı tanımlanması gerektiğinde kullanılır
  - Örnek: Departman-Proje-Sponsorluk bağıntısı üzerine Employee'nin monitor bağıntısı
  - Sponsorluk hadisesi üzerine monitor rolü tanımlanıyor
  - Eğer since ve until nitelikleri varsa (tarih aralığı), aggregation gerekli
  - Eğer sadece since varsa ve tek bir monitor varsa → ternary yeterli olabilir
- **UML Notasyonu**

  - Sınıf kutuları: İsim, nitelikler, operasyonlar (operasyon kullanılmıyor)
  - Association: Varlık setleri arası bağıntı (düz çizgi)
  - Generalization: Alt küme ilişkisi (üçgen ok)
  - Aggregation: Parça-bütün ilişkisi, parçalar tek başına anlamlı (içi boş elmas)
  - Composition: Parça-bütün ilişkisi, parçalar tek başına anlamsız (içi dolu elmas)
    - IR'daki zayıf varlığa denk gelir
  - Cardinality gösterimi: 1, 0..1, 0..*, 1..*, *
  - Existence durumu: 0..1 (olmayabilir), 1 (mutlaka olmalı)
- **Tasarım İyileştirme Kuralları**
  - Nitelik başka bir varlığı işaret ediyorsa → bağıntı haline dönüştür
  - Nitelik çok değer alabiliyorsa → varlık seti veya çoklu değerli nitelik haline dönüştür
  - Varlık seti sadece bir niteliğe sahip ve bir bağlantısı varsa → nitelik haline dönüştür
  - Genelleme/özelleştirme mümkünse uygulanmalı
  - Redundant (fazladan) bağıntı kontrolü yapılmalı
  - Aynı iki varlık seti arasında birden fazla bağıntı tanımlanabilir (farklı hadiseler için)
- **Company (Fabrika) Veritabanı Örneği**
  - Varlık setleri: Department, Project, Employee, Dependent
  - Bağıntılar:
    - Works_for: Employee → Department (N:1)
    - Manages: Employee → Department (1:1, start_date niteliği ile)
    - Works_on: Employee → Project (N:N, hours niteliği ile)
    - Controls: Department → Project (1:N)
    - Supervises: Employee → Employee (recursive, 1:N)
    - Dependent_of: Dependent → Employee (N:1, zayıf varlık)
  - Dependent zayıf varlık: Name ayırt edici nitelik
  - Number_of_departments → derived nitelik (SELECT COUNT ile hesaplanır)

Hocanın Özellikle Vurguladığı Kısımlar

- **Küçük dünyayı anlamak şart**: Ezbere tasarım yapılmaz, gereksinim listesi tahlil edilmeli
- **Cardinality belirlemede altın kural**: Diğer varlıklardan birer numune al, cümle kur, öyle olabilir mi düşünü
- **Zayıf varlık tanımı**: Anahtarı olmayan, güçlü varlığa var olma bağımlılığı olan varlık
- **Üçlü bağıntıların ikiliye dönüştürülmesi**: Her zaman mümkün değil, bilgi kaybı olabilir
- **Redundancy kontrolü**: Tasarım sonrası fazladan bağıntı var mı kontrol edilmeli
- **İki varlık seti arasında birden fazla bağıntı olabilir**: Farklı hadiseler için farklı bağıntılar
- **Derived nitelikler**: Statik olmayan bilgiler saklanmamalı, ihtiyaç oldukça hesaplanmalı
- **Tasarımın fiziksel etkisi**: Yanlış anahtar seçimi hantal sisteme yol açar
- **Notasyon kurallarına uyulmalı**: Çizim kuralları dışındaki çizim hatalı sayılır

Kısa Tekrar Notları

- ER tasarımı: Gereksinim toplama → Kavramsal şema → Gerçekleştirme modeli
- Varlık seti: Aynı niteliklere sahip varlıkların kümesi, dik dörtgen ile gösterilir
- Zayıf varlık: Çift çizgili dik dörtgen, anahtarı yoktur
- Bağıntı: Baklava şekli (eşkenar dörtgen), varlık setlerini bağlar
- Cardinality: 1 veya N değerleri, 4 kombinasyon (1:1, 1:N, N:N, N:1)
- Partial participation: Tek çizgi, Total participation: Çift çizgi
- Anahtar: Altı çizgili nitelik, varlıkları biricik kılar
- Super Key → Minimal Super Key → Candidate Key → Primary Key
- Ene enlik bağıntıda anahtar = iki tarafın anahtarları birlikte
- Bire enlik bağıntıda anahtar = N tarafının anahtarı
- Bire birlik bağıntıda anahtar = herhangi bir tarafın anahtarı
- Derived nitelik: Kesik çizgili elips, fiziksel olarak saklanmaz
- Multi-valued nitelik: Çift çizgili elips
- Composite nitelik: Alt dallara ayrılan elips

- Recursive bağıntı: Aynı varlık seti üzerinde, rol tanımlama gerekir
- Generalization/Specialization: Super type → Sub type, D/O kısıtlamaları
- Aggregation: Bağıntı üzerine bağıntı tanımlama
- UML: Association, Generalization, Aggregation, Composition
- Tasarım iyileştirme: Nitelik → Bağıntı, Nitelik → Varlık seti, Redundancy kontrolü

Detaylı Açıklamalar

- **Veritabanı Tasarım Felsefesi**: Veritabanı tasarımı kesin hatları belli bir disiplin değil, tecrübe ve meleke gerektiren bir alan. Küçük bir dünyayı çok farklı şekillerde modellemek mümkün, ancak her modelin fiziksel gerçeklemede farklı etkileri olacak. Tasarım sırasında bazı esaslara dikkat etmek, bazı hatalardan uzak durmak gerekiyor.
- **Cardinality Belirleme Yöntemi**: Cardinality belirlerken ezbere hareket edilmemeli. İlgili varlık setlerinin dışındaki varlıklardan birer numune alınarak bir cümle kurulmalı ve bu cümlenin mantıklı olup olmadığı düşünmelidir. Örneğin bir öğrenci bir günde bir saatte bir sınıfta bir derse kayıt olabilir → bu durumda o zaman dilimi için 1:N değil, 1:1 olur. Time değişkeni çıkarılırsa durum değişir.
- **Zayıf Varlık Mantığı**: Bir varlık setindeki tüm nitelikler bir araya gelse bile varlıkları biricik kılamıyorsa, o varlık seti zayıftır. Zayıf varlıklar güçlü bir varlığa bağımlıdır. Güçlü varlığın birincil anahtarı ile zayıf varlığın ayırt edici niteliği birlikte zayıf varlığın anahtarını oluşturur. Örneğin bir satış formundaki satırlar: Satır no tek başına biricik değildir çünkü her satış formunda 1, 2, 3… şeklinde tekrar eder. Ancak satış no + satır no birlikte biricik hale gelir.
- **Üçlü Bağıntı ve Aggregation İlişkisi**: Üçlü bağıntılar bazı durumlarda ikili bağıntılara dönüştürülebilir. Eğer bir bağıntı üzerine başka bir bağıntı tanımlanması gerekiyorsa (örn. bir bağıntının takip edilmesi/monitor edilmesi), aggregation kavramı devreye girer. Ancak bu durumda bilgi kaybı riski vardır. Zayıf varlık kullanarak üçlü bağıntı ikiliye dönüştürüldüğünde, üç güçlü varlığın anahtarları birleşerek zayıf varlığın anahtarını oluşturur ve tüm anlam korunur.
- **Tasarım İyileştirme Süreci**: ER diyagramı oluşturulduktan sonra redundancy kontrolü yapılmalı. Örneğin Grade Assignment örneğinde, eğer bir section sadece bir profesör tarafından veriliyorsa (1:1), assign bağıntısı redundant'dır ve çıkarılabilir. Ancak bir section birden çok profesör tarafından veriliyorsa (N:1), assign bağıntısı gereklidir. Bu tür kararlar küçük dünyanın detaylarına bağlıdır.
- **Derived Nitelik Kararları**: Yaş bilgisi her yıl değiştiği için saklanmamalı, doğum tarihi saklanıp yaş çıkarımla hesaplanmalıdır. Benzer şekilde bir departmandaki çalışan sayısı (number_of_employees) her ekleme/çıkarmada güncellenmesi gereken bir yük getirir. Bunun yerine SELECT COUNT(*) … GROUP BY ile ihtiyaç oldukça hesaplamak daha verimlidir. Statik olmayan bilgilerin saklanması gereksiz yük getirir.
- **Company Veritabanı Tasarımı**: Fabrika veritabanı örneğinde gereksinim listesinden varlık setleri ve bağıntılar adım adım çıkarılmıştır. Department, Project, Employee, Dependent varlık setleri; Works_for, Manages, Works_on, Controls, Supervises bağıntıları oluşturulmuştur. Dependent zayıf varlık olarak tanımlanmıştır çünkü istenen niteliklerle (isim, cinsiyet, doğum tarihi, ilişki) biricik anahtar seçilememiştir. Name, bir işçinin çocukları arasında ayırt edici nitelik olarak seçilmiştir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.